

# Retrieval Engine Competition Project

Notebook Report

Group: SeaFour

Maksym DOLHOV

Mehdi AGHAEI

Nguyen Ho Bao KHANH

**Report objective.** This version updates the written report to match the current saved notebook exactly, including its present configuration choices and its current evaluation protocol. The active configuration uses embedding first-stage retrieval, a cross-encoder second-stage reranker, and a TF-IDF + LinearSVC category classifier whose prediction is converted into a soft category bonus during reranking.

**Primary source files.** `kaggle-submissione.ipynb`, `solutions_SeaFour.csv`, and this L<sup>A</sup>T<sub>E</sub>X report.

|                               |                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------|
| <b>Notebook file</b>          | <code>kaggle-submissione.ipynb</code>                                                    |
| <b>Submission file</b>        | <code>solutions_SeaFour.csv</code>                                                       |
| <b>Current default model</b>  | <code>embedding</code>                                                                   |
| <b>Submission size target</b> | Top 7,500 first-stage candidates; offline rerank top 150; submission cell reranks top 20 |
| <b>Date</b>                   | March 27, 2026                                                                           |

# 1 Notebook-Aligned System Overview

The logic is organized into several stages: strict data loading, shared text normalization, sequential model experimentation (TF-IDF to BM25+ to embeddings), aggressive caching for performance, category-classifier fitting, cross-encoder training, offline evaluation on the training queries, and final submission generation.

The current system relies on a dense first-stage embedding retriever followed by a cross-encoder reranker, with category information injected as a soft bonus.

The present notebook snapshot is configured as follows:

- `EVALUATION_TOP_KS = [7500]`
- `CROSS_ENCODER_TRAIN_QUERY_LIMIT = 327`
- `CROSS_ENCODER_RERANK_TOP_M = 150` for offline evaluation
- category bonus = 2.0 whenever `ENABLE_CATEGORY_FILTER = True`
- the test-submission cell overrides the reranking depth to `top_m = 20`

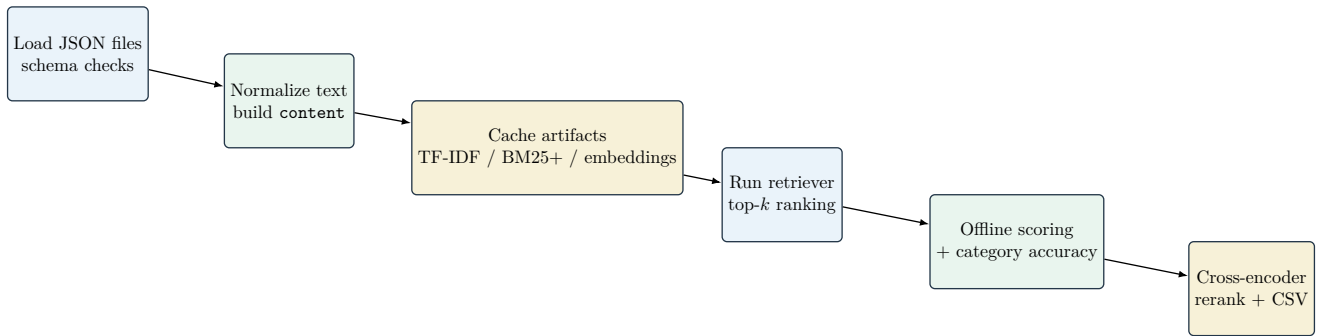


Figure 1: Active end-to-end workflow in the saved notebook.

| Component                              | Implemented | Active in saved run | Role in the pipeline                                                                                                                                                                             |
|----------------------------------------|-------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TF-IDF retrieval</b>                | Yes         | No                  | Sparse lexical baseline using cosine similarity. Fast, but struggled with semantic mismatch.                                                                                                     |
| <b>BM25+ retrieval</b>                 | Yes         | No                  | Stronger lexical baseline with token-level scoring and document-length normalization.                                                                                                            |
| <b>Embedding retrieval</b>             | Yes         | Yes                 | Dense semantic first-stage retrieval with Sentence-Transformers. Yielded the best metrics.                                                                                                       |
| <b>Cross-encoder reranker</b>          | Yes         | Yes                 | Re-ranks the first-stage candidates using joint query-document scoring. The current notebook uses $\text{top-}m = 150$ during offline evaluation and $\text{top-}m = 20$ in the submission cell. |
| <b>Category classifier</b>             | Yes         | Yes                 | Predicts one category per query, reports train-query accuracy, and provides a soft category bonus of 2.0 during reranking when enabled.                                                          |
| <b>Dominant-category strict filter</b> | Yes         | No                  | Kept as an implemented option, but disabled in the active saved run.                                                                                                                             |

Table 1: What the notebook supports versus what the saved execution actually uses.

## 2 Dataset and Preprocessing

The notebook loads 216,041 documents, 327 training queries, 141 test queries, and one training qrels file. The document collection is distributed across five categories, with significant class imbalance.

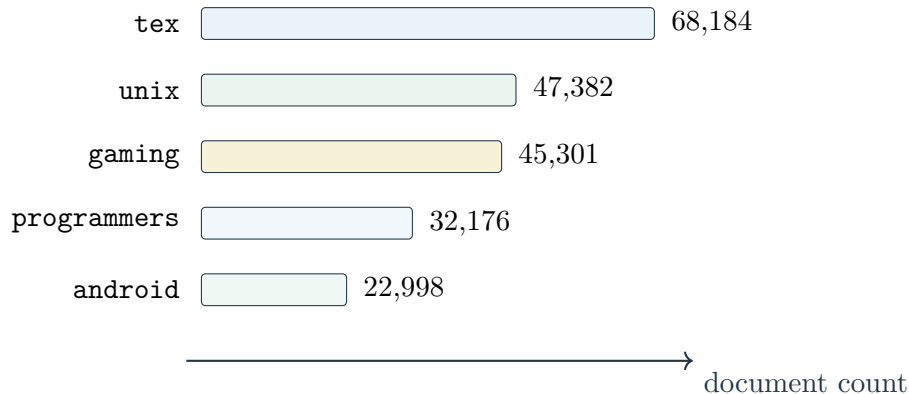


Figure 2: Document distribution by category.

Text normalization is shared across all components: separators `-`, `_`, and `/` are replaced by spaces, whitespace is collapsed, and text is lowercased. Retrieval documents are built from `title + text + tags`, while retrieval queries use only `title + text`. The reason is that document-side tags are strong, reliable topical signals already attached to the corpus, so including them enriches the searchable document representation at almost no risk. Query-side retrieval, however, is kept closer to the natural

user problem statement: title and body text carry the real retrieval intent, while query tags are handled separately in the classifier pipeline instead of being mixed directly into first-stage retrieval. This separation keeps the retrieval input closer to what the system will actually receive at test time while still allowing the classifier to exploit the extra categorical signal. The saved notebook prints average normalized lengths of 900.9 characters for documents, 48.4 for training queries, and 48.9 for test queries. This length disparity informed our ultimate shift away from purely lexical methods.

### 3 Experimental Progression and Pipeline Optimization

Our methodology evolved through a structured series of experiments, beginning with simple baselines and moving toward a complex, optimized semantic architecture.

#### 3.1 Phase 1 & 2: Lexical Baselines (TF-IDF and BM25+)

We initially implemented a **TF-IDF** retriever (using unigrams/bigrams with `min_df=2`). This served as a fast, lightweight baseline to validate our data loading and evaluation pipeline. While transparent, TF-IDF struggled significantly with semantic mismatch – when queries paraphrased document concepts without sharing exact vocabulary, recall dropped.

To improve this, we upgraded to **BM25+**. By tokenizing the text and applying `k1=1.5` and `b=0.75`, BM25+ provided much stronger lexical ranking, correcting for the high variance in document lengths. In the current notebook configuration, lexical components also use English stopwords filtering. However, BM25+ still fundamentally relied on exact token overlap, meaning synonymous phrasing remained a weakness.

#### 3.2 Phase 3: The Semantic Shift (Dense Embeddings)

To overcome the vocabulary mismatch problem, we transitioned to dense semantic retrieval using the Sentence-Transformer model `all-MiniLM-L6-v2`. Documents and queries were encoded into 384-dimensional L2-normalized vectors, allowing us to retrieve documents based on contextual meaning rather than exact keywords. This dramatically improved recall.

#### 3.3 Phase 4: Overcoming the 1.5-Hour Bottleneck

The shift to embeddings introduced a severe computational roadblock: initially, encoding the 216,041 documents and calculating scores took almost **1.5 hours to run**. This latency crippled our ability to run rapid iterations.

To solve this, we implemented aggressive caching and chunking mechanisms:

- **Disk Caching:** We engineered a caching system that fingerprints the dataframes and saves the expensive document embeddings as `.numpy` arrays. Repeated notebook runs now load the matrix in seconds instead of re-encoding.
- **Chunked Query Scoring:** Instead of creating massive, memory-crashing score blocks, we scored queries in chunks of 32 (`EMBEDDING_QUERY_CHUNK_SIZE = 32`).
- **Partial Sorting:** We replaced full array sorts with `np.argpartition` to extract only the top-*K* candidates efficiently.

#### 3.4 Phase 5: Current Two-Stage Reranking Configuration

With the dense pipeline running smoothly, the notebook fixes the first-stage retrieval depth to `top_k = 7,500` for both offline evaluation and submission generation. The current notebook snapshot eval-

uates only the `embedding` retriever (`EVALUATION_MODELS = ("embedding",)`) and stores one offline configuration table for that model.

To refine the first-stage ranking, the notebook adds two auxiliary components:

- a TF-IDF + LinearSVC category classifier trained on document-side category labels;
- a cross-encoder reranker trained on all 327 training queries.

More specifically:

- the classifier training frame is `docs_classifier_df`;
- classifier content is built from `title + text + tags`;
- query-side classifier frames also use `title + text + tags`;
- the retriever itself continues to use only `title + text`;
- `CROSS_ENCODER_TRAIN_QUERY_LIMIT = 327`;
- `CROSS_ENCODER_HARD_NEGATIVE_TOP_K = 200`;
- up to four positives per query and four negatives per positive are used for cross-encoder training;
- offline reranking uses `CROSS_ENCODER_RERANK_TOP_M = 150`;
- the submission cell overrides reranking depth and uses only the top 20 documents per query.

The category signal is no longer applied as a strict hard filter inside the active offline loop. Instead, the notebook computes

$$\text{current\_bonus} = \begin{cases} 2.0 & \text{if ENABLE\_CATEGORY\_FILTER} = \text{True} \\ 0.0 & \text{otherwise} \end{cases}$$

and adds that scalar bonus to cross-encoder scores for documents whose category matches the predicted query category.

## 4 Evaluation Protocol and Results

The current saved notebook does *not* build a held-out validation split. Instead, it performs the following exact sequence:

1. fit the TF-IDF + LinearSVC category classifier on `docs_classifier_df`, the document corpus with category labels,
2. compute category predictions for the training-query and test-query classifier frames, then report `classifier_accuracy` on the training queries,
3. train or load the cross-encoder on `train_queries_df` with `CROSS_ENCODER_TRAIN_QUERY_LIMIT = 327`,
4. run retrieval on `train_queries_df` at `top_k = 7,500`,
5. rerank the top 150 documents per query with the cross-encoder and the soft category bonus,
6. score the resulting ranking against `ground_truth` for the same 327 training queries.

So the present notebook's offline score is a **training-query score**. It is aligned with the saved notebook code, but it is not a held-out validation estimate. This distinction matters when interpreting the resulting numbers. One older markdown explanation inside the notebook still says the system writes the first-stage ranking directly; however, the active saved code and outputs clearly include cross-encoder reranking and the category bonus. This report follows the executed pipeline rather than that stale prose.

The combined score is calculated as:

$$\text{Score} = \frac{1}{4} (\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy})$$

| Setting                             | Current notebook value                                                       |
|-------------------------------------|------------------------------------------------------------------------------|
| <b>EVALUATION_MODELS</b>            | ("embedding",)                                                               |
| <b>EVALUATION_TOP_KS</b>            | [7500]                                                                       |
| <b>Offline query set</b>            | <code>train_queries_df</code> (327 training queries)                         |
| <b>Cross-encoder train scope</b>    | all 327 training queries; <code>CROSS_ENCODER_TRAIN_QUERY_LIMIT = 327</code> |
| <b>Offline rerank depth</b>         | <code>CROSS_ENCODER_RERANK_TOP_M = 150</code>                                |
| <b>Submission rerank depth</b>      | hard-coded <code>rerank_top_m = 20</code> in the submission cell             |
| <b>Category bonus</b>               | 2.0 when <code>ENABLE_CATEGORY_FILTER = True</code>                          |
| <b>Reported classifier accuracy</b> | computed on the training queries                                             |

Table 2: Evaluation and reranking settings in the current saved notebook.

The notebook prints metric values dynamically from `leaderboard_score(...)` and stores the best row in `best_experiment = eval_summary_df.iloc[0]`. Because those numbers are not hard-coded in the source file and because the current protocol evaluates on the training queries, this report documents the protocol and settings rather than claiming a fresh held-out score.

## 5 Conclusion

Our system evolved from basic lexical matching to a two-stage semantic retrieval pipeline centered on dense embeddings, aggressive caching, and cross-encoder reranking. In the current saved notebook snapshot, the active configuration is very specific: embedding-only first-stage retrieval at `top_k=7,500`, a cross-encoder trained on all 327 training queries, a soft category bonus of 2.0 when enabled, offline reranking of the top 150 candidates, and a submission-cell override that reranks only the top 20 test candidates. This report now matches that notebook behavior directly, including the important fact that the present offline score is computed on the training query set rather than on a separate validation split.